

HR Buddy

Asistente Virtual de RR.HH. para ChocolaTech

Curso de Inmersión en Agentes IA
Documentación Técnica del Proyecto

Sebastián Salazar

5 de junio de 2026

Índice

1. Descripción General del Proyecto	2
2. Arquitectura del Sistema	2
2.1. Flujo principal en n8n	2
2.2. Componentes del Agente	2
3. Implementación de RAG	3
3.1. Proceso de vectorización	3
3.2. Uso en consultas	3
4. Base de Datos — MySQL en Railway	3
4.1. Estructura de la tabla	3
5. Prompt del Agente	3
6. Desafío 1 — Filtro de Mensajes No Relevantes	4
6.1. Problema identificado	4
6.2. Solución implementada — Router Pattern	5
6.3. Prompt del clasificador	5
6.4. Configuración del Switch	5
6.5. Resultado	5
7. Desafío 2 — Seguridad e Identidad del Empleado	6
7.1. Problema identificado	6
7.2. Solución implementada — Autenticación por Telegram ID	6
7.2.1. Paso 1 — Modificación de la base de datos	6
7.2.2. Paso 2 — Extracción del ID desde Telegram Trigger	6
7.2.3. Paso 3 — Modificación del nodo MySQL	7
7.3. Comparativa de seguridad	7
7.4. Resultado	7
8. Tecnologías Utilizadas	7
9. Conclusiones	7

1. Descripción General del Proyecto

HR Buddy es un chatbot de Recursos Humanos desarrollado como proyecto práctico del Curso de Inmersión en Agentes IA. Fue construido sobre la plataforma **n8n** e integra múltiples tecnologías para responder consultas de empleados de la empresa ficticia *ChocolaTech*.

El sistema combina:

- Recuperación aumentada por generación (**RAG**) sobre documentos internos
- Un agente de IA con modelo **Cohere**
- Base de datos relacional **MySQL** alojada en Railway
- Interfaz de usuario vía **Telegram**

2. Arquitectura del Sistema

2.1. Flujo principal en n8n

El flujo general del sistema sigue esta secuencia:

1. **Telegram Trigger** — recibe el mensaje del usuario
2. **Basic LLM Chain** — clasifica el mensaje como RRHH o GENERICO
3. **Switch** — enruta el flujo según la clasificación
 - Ruta RRHH → AI Agent completo
 - Ruta GENERICO → respuesta de texto fija
4. **AI Agent** — procesa la consulta usando herramientas
5. **Send a Text Message** — responde al usuario en Telegram

2.2. Componentes del Agente

Componente	Descripción
Cohere Chat Model	Modelo principal del agente para generación de respuestas
Simple Memory	Memoria de conversación para mantener contexto
Simple Vector Store	Almacén vectorial con políticas internas de RR.HH.
Embeddings Cohere	Generación de embeddings para búsqueda semántica
MySQL Tool	Consulta de datos personales de empleados en Railway

3. Implementación de RAG

3.1. Proceso de vectorización

El documento con las políticas internas de ChocolaTech fue procesado mediante:

1. **Carga** del documento de texto plano
2. **Chunking** — división en fragmentos optimizados
3. **Embedding** — vectorización con Cohere Embeddings
4. **Almacenamiento** en Simple Vector Store

3.2. Uso en consultas

Cuando el usuario realiza una consulta general sobre políticas de RR.HH., el agente realiza una búsqueda semántica en el Vector Store y responde con base en el contenido relevante recuperado, sin inventar información.

4. Base de Datos — MySQL en Railway

4.1. Estructura de la tabla

Listing 1: Definición de la tabla empleados

```
1 CREATE TABLE empleados (  
2     id                INT AUTO_INCREMENT PRIMARY KEY,  
3     nombre            VARCHAR(100)      NOT NULL,  
4     email             VARCHAR(150)      NOT NULL UNIQUE,  
5     departamento      VARCHAR(100)      NOT NULL,  
6     puesto            VARCHAR(100)      NOT NULL,  
7     fecha_ingreso     DATE              NOT NULL,  
8     saldo_vacaciones  INT              NOT NULL DEFAULT 0,  
9     banco_horas       DECIMAL(5,1)      NOT NULL DEFAULT 0,  
10    modalidad         VARCHAR(20)       NOT NULL DEFAULT 'hibrido',  
11    telegram_id       BIGINT            UNIQUE DEFAULT NULL  
12 );
```

La columna `telegram_id` fue añadida posteriormente como mejora de seguridad (ver Sección 7).

5. Prompt del Agente

El comportamiento del agente está definido por el siguiente system prompt:

Eres el HR Buddy, asistente virtual de RR.HH. de ChocolaTech.

REGLAS:

- Responde siempre en español.
- Responde SOLO dudas relacionadas con RR.HH.

IDENTIFICACIÓN DEL EMPLEADO:

El telegram_id del usuario actual es:

```
{{ $('Telegram Trigger').item.json.message.from.id }}
```

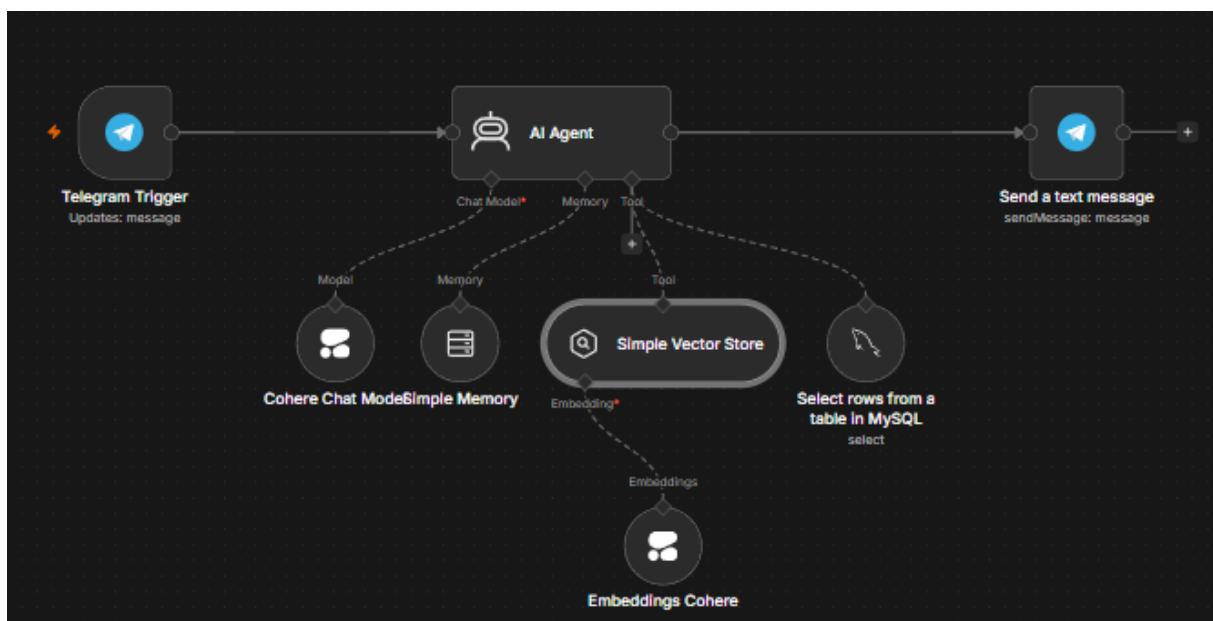
Usa la herramienta MySQL para buscar en la tabla empleados donde telegram_id sea exactamente igual al valor indicado.

- NO le preguntes el nombre ni ningún dato al usuario.
- NO uses ningún número o nombre que el usuario escriba.
- SIEMPRE usa el telegram_id del contexto indicado arriba.

Si se encuentra el registro: responde usando los datos reales del empleado (nombre, saldo_vacaciones, banco_horas, etc).

Si NO se encuentra: informa que su cuenta de Telegram no está vinculada a ningún empleado de ChocolaTech y que debe contactar a RR.HH. para registrarse.

Usa la base de conocimientos (Vector Store) solo para dudas generales de políticas de RR.HH.



6. Desafío 1 — Filtro de Mensajes No Relevantes

6.1. Problema identificado

El agente procesaba **todos** los mensajes entrantes con el modelo completo de Cohere, incluyendo saludos, bromas y consultas no relacionadas con RR.HH., generando un consumo innecesario de tokens.

6.2. Solución implementada — Router Pattern

Se implementó un patrón de enrutamiento previo al agente:

1. Se añadió un nodo **Basic LLM Chain** con un modelo liviano entre el Telegram Trigger y el AI Agent
2. El clasificador recibe el mensaje y responde únicamente con RRHH o GENERICO
3. El nodo **Switch** lee el output y desvía el flujo

6.3. Prompt del clasificador

Clasifica el siguiente mensaje en UNA sola palabra.
Responde ÚNICAMENTE con una de estas dos opciones:

- RRHH: si el mensaje está relacionado con recursos humanos, vacaciones, permisos, nómina, contratos, horarios o cualquier tema laboral.
- GENERICO: si es un saludo, broma, pregunta general, tema no laboral, o cualquier otra cosa.

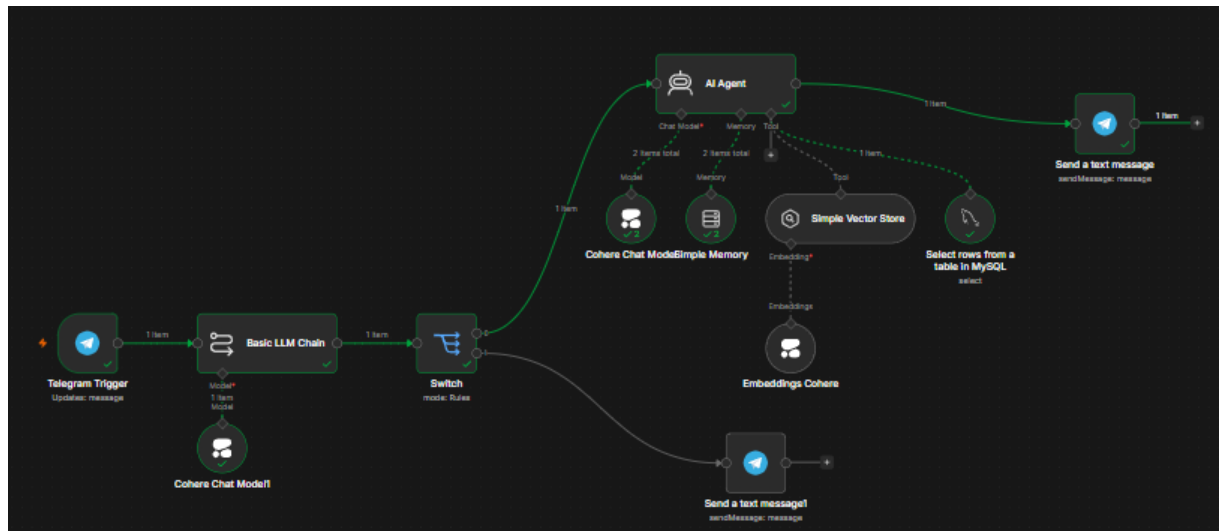
No expliques nada. Solo responde: RRHH o GENERICO.

6.4. Configuración del Switch

Regla	Valor	Destino
Rule 1	<code>\$json.text.trim().toUpperCase()</code> AI Agent contiene RRHH	
Rule 2	<code>\$json.text.trim().toUpperCase()</code> Texto fijo contiene GENERICO	

6.5. Resultado

- Ahorro del **100 % de tokens** del agente en mensajes irrelevantes
- Respuestas instantáneas para mensajes genéricos
- El agente solo se invoca cuando es estrictamente necesario



7. Desafío 2 — Seguridad e Identidad del Empleado

7.1. Problema identificado

El agente identificaba al empleado por el **nombre que el usuario declaraba** en el chat. Esto permitía que cualquier persona escribiera el nombre de otro empleado y accediera a sus datos personales, representando una vulnerabilidad crítica de seguridad.

7.2. Solución implementada — Autenticación por Telegram ID

Se reemplazó la identificación por nombre con el **telegram_id**, un identificador único e inmutable asignado por Telegram que el usuario no puede falsificar.

7.2.1. Paso 1 — Modificación de la base de datos

Listing 2: Adición de columna telegram_id

```
1 ALTER TABLE empleados
2 ADD COLUMN telegram_id BIGINT UNIQUE DEFAULT NULL;
3
4 -- Ejemplo de registro de un empleado
5 UPDATE empleados
6 SET telegram_id = 7171173451
7 WHERE nombre = 'Juan Silva';
```

7.2.2. Paso 2 — Extracción del ID desde Telegram Trigger

El ID real del usuario se obtiene directamente del payload de Telegram:

Listing 3: Expresión n8n para obtener el telegram_id

```
1 {{ $('Telegram Trigger').item.json.message.from.id }}
```

Este valor es inyectado directamente en el prompt del agente y en la consulta MySQL.

7.2.3. Paso 3 — Modificación del nodo MySQL

La búsqueda cambió de:

```
1 -- Antes (inseguro)
2 SELECT * FROM empleados WHERE nombre LIKE '%nombre_del_usuario%'
```

```
1 -- Después (seguro)
2 SELECT * FROM empleados
3 WHERE telegram_id = [valor inyectado desde Telegram Trigger]
```

7.3. Comparativa de seguridad

Antes	Después
Busca por nombre declarado por el usuario	Busca por ID certificado por Telegram
Cualquiera puede suplantar a otro empleado	Imposible falsificar el telegram_id
El agente pregunta quién es el usuario	El agente ya sabe quién es
Vulnerabilidad crítica de suplantación	Identidad verificada por la plataforma

7.4. Resultado

El agente ignora completamente cualquier nombre o número que el usuario escriba en el chat. La identidad es certificada por Telegram, eliminando por completo la posibilidad de suplantación.

8. Tecnologías Utilizadas

Tecnología	Uso
n8n	Orquestación del flujo completo
Cohere	Modelo de lenguaje (agente y clasificador)
MySQL / Railway	Base de datos de empleados
Telegram Bot API	Interfaz de usuario del chatbot
Simple Vector Store	Almacenamiento de embeddings RAG

9. Conclusiones

Este proyecto demostró la viabilidad de construir un agente de RR.HH. funcional y seguro usando herramientas de bajo código como n8n. Los dos desafíos abordados representan problemas reales en producción:

- El **router pattern** es una técnica aplicable a cualquier chatbot para reducir costos operativos sin sacrificar calidad

- La **autenticación por ID de plataforma** es una práctica de seguridad esencial en sistemas que manejan datos personales

Ambas soluciones son escalables y pueden adaptarse a otros proyectos de agentes conversacionales en contextos empresariales reales.